Bachelor's thesis

Bachelor's Programme in Computer Science

Large Language Model-based Methods for Phishing Email Detection

Tony Kuosa

February 11, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki, Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Bachelor's Programme in Computer Science

Tekijä — Författare — Author

Tony Kuosa

Työn nimi — Arbetets titel — Title

Large Language Model-based Methods for Phishing Email Detection

Ohjaajat — Handledare — Supervisors

Dr. Nasrulloh Loka, Prof. Luigi Acerbi

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Bachelor's thesis	February 11, 2026	31 pages

Tiivistelmä — Referat — Abstract

Despite decades of research on countermeasures, phishing attacks continue to inflict significant harm on individuals and organizations. Current methods of phishing detection struggle to keep up with continuously evolving attacks, and an additional challenge is presented by the emergence of large language model (LLM)-generated or rephrased phishing attacks.

To address these issues, we investigate the use of LLMs: with their advanced capabilities in natural language processing (NLP) tasks, LLMs have the potential to reshape the domain of phishing, both in attack and defense. We specifically focus on attacks occurring via email, a typical medium for phishing.

We find results suggesting that LLM-based phishing email detection methods may outperform baseline machine learning and deep learning methods, and that multi-agent solutions may outperform single-agent approaches with and without chain-of-thought reasoning. Within multi-agent systems, mixed-model configurations may outperform same-model configurations. The impact of prompt engineering techniques, including chain-of-thought reasoning and role prompting, is found to be inconclusive.

We identify several challenges, including the use of benchmark datasets that do not adequately reflect present-day phishing campaigns, limited consideration of the threat posed by LLM-generated phishing, and high computational costs associated with LLM-based detection methods. Based on these findings, we present recommendations and directions for future research.

ACM Computing Classification System (CCS)
Security and privacy → Intrusion/anomaly detection and malware mitigation → Social engineering attacks → Phishing
Computing methodologies → Artificial intelligence → Natural language processing

Avainsanat — Nyckelord — Keywords

large language models, phishing email detection

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Lyhennelmä

Tietojenkalastelu on vuosikymmenten tutkimustyöstä huolimatta edelleen merkittävä kyberuhka yksilöille ja yrityksille. Tietojenkalasteluhyökkäyksellä tarkoitetaan tilannetta, missä hyökkääjä pyrkii vilpillisin keinoin saamaan uhrin paljastamaan arkaluonteista tietoa, kuten kirjautumistunnuksia, henkilötietoja tai pankkitunnuksia, tyypillisesti tietojenkalastelusivuston (esim. väärennetty pankin kirjautumissivu) tai haittaohjelman (esim. laskuksi tekaistu haitallinen tiedosto) välityksellä [5]. Saatua tietoa käytetään hyväksi esimerkiksi identiteettivarkauksiin sekä petos- ja kiristysrikoksiin [23, 17]. Uhri saattaa seurauksena kärsiä taloudellista vahinkoa ja mainehaittaa [20].

Hyökkääjät esiintyvät tyypillisesti uhrille tuttuna ja luotettavana tahona (esim. työkollega, pankki tai palveluntarjoaja). Hoxhuntin vuoden 2025 selvityksen [5] mukaan hyökkääjien useimmin imitoimia tahoja ovat Microsoft, Docusign ja organisaatioiden henkilöstöosastot. Yleisimpiä väyliä tietojenkalastelulle on sähköposti: hyökkääjät lähettävät muun muassa väärennettyjä Microsoft Office -sähköpostiviestejä [47], jotka ohjaavat uhrin jonkin tekosyyn (esim. jaetun asiakirjan tarkastelu) varjolla navigoimaan viestissä olevan linkin kautta Office-palvelun kirjautumissivua matkivalle tietojenkalastelusivustolle. Tietojenkalasteluviesteissä käytetään tyypillisesti kiireellistä tai uhkailevaa kieltä, ja uhria saatetaan kehottaa suorittamaan jokin kirjautumista vaativa toimenpide (esim. vanhentuvan salasanan vaihto tai uudistuneiden käyttöehtojen hyväksyminen) lyhyen ajan sisällä, tai tili, jota viesti koskee, jäädytetään [23].

Tietojenkalastelu on erittäin helppopääsyinen kyberrikos: hyökkääjien suosiossa ovat pimeässä verkossa edulliseen hintaan myytävät käyttövalmiit tietojenkalastelupakkaukset (phishing kits), jotka mahdollistavat tietojenkalastelukampanjoiden aloittamisen ilman teknistä osaamista [5]. Saavutettavuutensa vuoksi tietojenkalastelu onkin yksi yleisimpiä, ellei jopa yleisin kyberrikos. IBM:n vuoden 2025 selvityksessä [20] todetaan tietojenkalastelun olleen yleisin hyökkäysvektori yritykseen kohdistuneelle tietomurrolle, aiheuttaen keskimäärin 4,8 miljoonan Yhdysvaltain dollarin arvosta vahinkoa.

Tällä hetkellä tietojenkalastelusähköpostien havaitsemiseen käytetään muun muassa lista- ja sääntöpohjaista estämistä, tilastollisia menetelmiä ja kone- sekä syväoppimiseen perustuvia menetelmiä. Nämä keinot eivät kuitenkaan ole täysin ratkaisseet tietojenkalastelun ongelmaa, sillä osa tietojenkalastelusta pääsee edelleen suodattimien läpi

[5]. Nousevana huolenaiheena on lisäksi laajoilla kielimalleilla tuotettu tai uudelleenmuotoiltu tietojenkalastelu, jonka havaitsemisessa tarjolla olevat menetelmät, kuten Gmailin roskapostisuodatin [24], Apache SpamAssassin [3], tukivektorikone, logistinen regressio ja naiivi Bayes ovat pärjänneet heikommin verrattuna perinteiseen tietojenkalastelusähköpostiin [1].

Vaikka laajojen kielimallien kyvyt luonnollisen kielen käsittelyn tehtävissä tekevät niistä huolestuttavan apuvälineen hyökkääjille, samat kyvyt tekevät niistä lupaavia ehdokkaita myös tietojenkalastelusähköpostien havaitsemiseen. Edellä mainittujen seikkojen motivoimana tämän tutkielman tavoitteena on selvittää laajojen kielimallien käyttöä tietojenkalastelusähköpostien havaitsemisessa analysoimalla aiheesta tehtyä tutkimusta. Pyrkimyksenä on tarkastella esitettyjä laajoihin kielimalleihin perustuvia menetelmiä, ymmärtää niiden impakti ja tunnistaa mahdollisia rajoitteita, ongelmia sekä tulevaisuuden tutkimussuuntia.

Laajat kielimallit ovat kielimalleja, jotka on koulutettu suurella määrällä tekstiä. Kielimalleja käytetään tyypillisesti tekstin tuottamiseen, ja niiden käyttötarkoituksia ovat muun muassa tekstin luokittelu, kysymyksiin vastaaminen ja kielikäännösten teko. Kielimallille annetaan syötteenä kehote (prompt), johon se tuottaa vastauksen arvaamalla toistuvasti edellisten sanojen* perusteella seuraavana todennäköisesti olevan sanan [21]. Yksinkertaisen kielimallin konteksti saattaa koostua muutamasta edellisestä sanasta, kun taas laajojen kielimallien kontekstikoko (context size) voi olla kymmeniä tuhansia sanoja, mikä tarjoaa merkittävästi enemmän tietoa arvausten tekemiseen.

Laajan kielimallin arvaukset määrittyvät sen koulutuksen aikana säädettyjen satojen miljardien parametrien eli painojen (weights) perusteella [21]. Painoja säädetään kouluttaessa siten, että kielimalli kykenee arvaamaan koulutusaineiston tekstiotteessa seuraavaksi esiintyviä sanoja oikein. Painoja saatetaan esikoulutusvaiheen jälkeen lisäksi vielä hienosäätää muun muassa ihmispalautteen perusteella halutunlaisten vastausten tuottamiseksi. Kun laaja kielimalli tuottaa vastauksen annettuun kehotteeseen, se käytännössä arvaa koulutuksensa pohjalta, miltä haluttu vastaus voisi mahdollisesti näyttää.

Laajat kielimallit perustuvat muuntaja-arkkitehtuuriin (transformer architecture), joka on huomiomekanismiin (attention mechanism) perustava neuroverkkoarkkitehtuuri

*Tarkemmin kuvattuna kielimalli käsittelee tokeneiksi muutettuja sanoja: token voi olla esimerkiksi kokonainen sana, sanan osa tai välimerkki [21]. Tutkielmassa tokeneihin kuitenkin viitataan sanoina yksinkertaisuuden vuoksi.

[50]. Aiempien kielimallien kohdalla tiedon peräkkäinen käsittely on toiminut huomattavana laskennallisena rajoitteena. Muuntaja-arkkitehtuurin esittely olikin tärkeä edistysaskel kielimalleille: tiedon rinnakkainen käsittely vähentää merkittävästi mallin koulutusaikaa, mikä puolestaan mahdollistaa suuremmat koulutusaineistot, kontekstikoot ja parametrien määrät. Huomiomekanismi määrittää, miten tekstissä olevien sanojen merkitys päivittyy niitä ympäröivien sanojen perusteella. Tietyn sanan kohdalla määritetään, mitkä sanat ovat sen merkityksen kannalta olennaisia ja miten merkitystä tämän perusteella päivitetään.

Tietojenkalastelusähköpostien havaitseminen on binääriluokitustehtävä, jossa tehtävänä on luokitella annettu sähköposti tietojenkalasteluksi (phishing) tai legitiimiksi (legitimate). Erityisen tärkeää on minimoida väärät negatiiviset luokitukset, eli tapaukset, joissa tietojenkalastelusähköposti luokitellaan virheellisesti legitiimiksi. Toivottavaa on myös pitää väärien positiivisten määrä pienenä, eli välttää tapaukset, joissa legitiimi sähköposti luokitellaan virheellisesti tietojenkalasteluksi, mutta kriittisintä on estää tietojenkalastelusähköpostia tavoittamasta käyttäjää [55].

Laajojen kielimallien tuottamien vastausten muokkaamiseen käytetään usein kehoteohjelmointitekniikoita (prompt engineering techniques) [54]. Mallia voidaan esimerkiksi ohjeistaa tuottamaan vastauksia tietyllä rakenteella. Laajoihin kielimalleihin perustuvat tietojenkalastelusähköpostien havaitsemismenetelmät palauttavat tyypillisesti raportin, joka sisältää sähköpostin luokituksen, perustelut ja luottamusarvon (confidence score): lisätyn tiedon tavoitteena on auttaa käyttäjää paremmin ymmärtämään ja arvioimaan sähköpostin luonnetta. Mallin vastausten laadun ja tarkkuuden parantamiseksi kokeillaan usein ketjutettua päättelyä (chain-of-thought reasoning) ja rooliohjelmointia (role prompting) [22].

Tutkielmassa tarkastellaan kolmea laajoihin kielimalleihin perustuvaa menetelmää tietojenkalastelusähköpostien havaitsemiseen: Koide et al. [22], Nguyen et al. [51] ja Xue et al. [55].

Koide et al. [22] esittämässä järjestelmässä sähköpostia arvioi yksittäinen kielimalliagentti. Arvioituna on neljä eri laajaa kielimallia: GPT-3.5-Turbo [7], GPT-4 [33], Llama 2-70B [48] ja Gemini 1.0 Pro [46]. Käytössä on yksinkertainen ja normaali kehote, joista jälkimmäinen hyödyntää kehoteohjelmointitekniikoita (päättelyn ketjutus, rooliohjaus). Järjestelmä palauttaa luokituksen, perustelut, ja luottamusarvon [22].

Nguyen et al. [51] esittämässä järjestelmässä sähköpostia arvioi kolme kielimalliagenttia ennalta määritetyillä rooleilla. Arvioituna on kaksi laajaa kielimallia, GPT-4 ja

Llama 2. Kaksi agenttia väittelee sähköpostin luokituksesta kahden kierroksen ajan, joiden päätteeksi tuomarina toimiva kolmas agentti arvioi esitetyt argumentit ja tekee lopullisen päätöksen. Lisäksi arvioidaan ketjutetun päättelyn ja rooliohjelmoinnin käyttöä. Järjestelmä palauttaa luokituksen ja perustelut.

Xue et al. [55] esittämässä järjestelmässä sähköpostia arvioi viiden kielimalliagentin kokonaisuus. Käytettynä laajana kielimallina toimii GPT-4o [34]. Ensimmäinen agentti toimii vastapuoliagenttina, joka tuottaa sähköpostiaineiston näytteistä uudelleenkirjoitettuja variantteja [55]. Sähköpostin sisältöä tutkii kolme agenttia omine osa-alueineen (teksti, URL, metadata): kukin palauttaa oman luokituksen, perustelut ja luottamusarvon. Analyysiagenttien vastausten painot (eli tärkeys) säätyvät sähköpostista poimittujen ominaisuuksien perusteella. Järjestelmä palauttaa painojen määrittämän lopullisen luokituksen sekä yksinkertaistaja-agentin koostaman perustelujen yhteenvedon.

Kuten aiemmin mainittu, tietojenkalastelusähköpostien havaitsemisen kannalta kriittistä on minimoida väärien negatiivisten luokitusten määrä: tällöin keskeisiä arviointimittareita ovat saanti (recall) ja väärien negatiivisten osuus (false negative rate). Näiden lisäksi tarkastellaan täydentävänä mittarina tarkkuutta (accuracy), joka mittaa menetelmän yleistä kykyä luokitella sähköposteja tietojenkalasteluksi ja legitiimiksi. Koide et al. [22] ja Xue et al. [55] ilmoittamat tulokset viittaavat esitettyjen järjestelmien pärjänneen hyvin tietojenkalastelusähköpostien havaitsemisessa, saavuttaen korkean saannin, matalan väärien negatiivisten osuuden ja korkean tarkkuuden. Nguyen et al. [51] tuloksissa ei sen sijaan ilmoitettu saantia tai väärien negatiivisten osuutta, mikä vaikeuttaa esitetyn järjestelmän arviointia. Huomautettakoon, että raportoidut tulokset eivät ole keskenään vertailukelpoisia, sillä tulokset on saatu käyttäen erilaisia testiaineistoja.

Koide et al. [22] tulokset osoittavat laajojen kielimallien pärjänneen kone- ja syväoppimisen vertailumenetelmiä (satunnaismetsä [6], tukivektorikone [26], neuroverkko [30] ja konvoluutioneuroverkko [13]) paremmin. Vertailumenetelmistä neuro- ja konvoluutioverkko olivat kuitenkin koulutettu yleisen roskapostin havaitsemiseen, mikä saattoi asettaa ne epäedulliseen asetelmaan arvioitaessa nimenomaisesti tietojenkalastelun havaitsemiskykyä. Parhaiten laajoista kielimalleista pärjäsi GPT-4 erinomaisella 99,7% saannilla, 0.3% väärien negatiivisten osuudella ja 99.7% tarkkuudella. Normaalin kehotteen käyttö hieman paransi GPT-4:n suorituskykyä kaikilla mittareilla. Muiden mallien kohdalla normaali kehote vähensi väärien negatiivisten luokitusten määrää ja lisäsi väärien positiivisten määrää, eli mallit luokittelivat sähköposteja aggressiivisem-

min. Koska erityisen tärkeää on väärien negatiivisten minimointi, kehoteohjelmointitekniikoiden käytön voidaan katsoa olleen hyödyksi.

Nguyen et al. [51] tulokset osoittavat erilaisia malleja (mixed-model) yhdistelevien kokoonpanojen päihittäneen saman mallin (same-model) kokoonpanot, mikä saattaa viitata mallien yhdistelyn hyötyihin usean agentin järjestelmissä. Kuten mainittu, tuloksissa ei ollut ilmoitettu havaitsematta jääneiden tietojenkalastelusähköpostien osuutta ilmaisevaa saantia tai väärien negatiivisten osuutta, mikä vaikeuttaa järjestelmän suorituskyvyn arvioimista nimenomaisesti tietojenkalastelusähköpostien havaitsemisessa. Raportoitu tarkkuus, 99,1%, oli kuitenkin korkea. Kehoteohjelmointitekniikoiden käyttö ei tässä asetelmassa Koide et al. [22] raportoimista tuloksista poiketen tuottanut parempia tuloksia. Tätä voisivat selittää esimerkiksi erot agenttien määrissä (usean vs. yksittäisen agentin käyttö) sekä järjestelmän valmiiksi jäsennelty rakenne, missä tehtävän ratkominen on jo jaettu kahdelle väittelykierrokselle ja kolmelle agentille [51].

Xue et al. [55] tulokset osoittavat viiden agentin järjestelmän päihittäneen yksittäisen agentin, yksittäisen agentin ketjutetulla päättelyllä ja RoBERTan [25] eli hienosäädetyn BERT-kielimallin. Usean agentin järjestelmä saavutti 99.80% saannin, 0.2% väärien negatiivisten osuuden ja 97.89 tarkkuuden%. Yksittäinen agentti ketjutetulla päättelyllä tuotti hieman enemmän vääriä negatiivisia verrattuna yksittäiseen agenttiin ilman ketjutettua päättelyä, vaikkakin ketjutetun päättelyn agentin tarkkuus oli hieman korkeampi. Ketjutettu päättely siis hieman lisäsi havaitsematta jääneen tietojenkalastelun osuutta, joten sen käytön voidaan tässä tapauksessa todeta olleen haitaksi.

Käytettyjen testiaineistojen tarkastelu paljastaa niiden olevan laadultaan vaihtelevia. Koide et al. [22] käyttämä testiaineisto oli sisällöltään yleisesti varsin laadukasta ja kattavaa: sähköpostinäytteet ovat varsin tuoreita (2021–2023), monikielisiä (tietojenkalastelua 19 eri kielellä, legitiimiä 12 eri kielellä) eikä tietojenkalastelunäytteiden kanssa ollut sekoitettu yleisluontoista roskapostia. Aineistojen valinnassa ei kuitenkaan ollut otettu huomioon laajoilla kielimalleilla tuotettua tietojenkalastelua.

Nguyen et al. [51] käyttämä testiaineisto on laadultaan heikompaa: näytteistä suuri osa on kerätty ennen vuotta 2010, ne ovat täysin yksikielisiä (englanti) ja tietojenkalastelunäytteiden kanssa on sekoitettu yleisluontoista roskapostia. Tämä tarkoittaa, ettei järjestelmien suorituskykyä mitattu pelkästään tietojenkalastelun tunnistamisessa, kuten tutkimusten tavoitteena oli. Tämä herättää myös kysymyksen: onko mielekästä mitata mallin suorituskykyä sähköpostiaineistolla, joka on yli 15 vuotta vanhaa, yksikielistä ja sisältää yleistä roskapostia? Voidaan todeta, ettei aineistolla mitattu suori-

tuskyky anna kattavaa kuvaa mallien todellisesta suorituskyvystä. Pieni osa näytteistä oli keinotekoisesti tuotettuja, mutta tuottamiseen käytettyä menetelmää ei ollut kerrottu. Jää siis epäselväksi, huomioitiinko testiaineiston näytteiden valinnassa laajoilla kielimalleilla tuotettua tietojenkalastelua.

Xue et al. [55] käyttämä testiaineisto on niin ikään laadultaan heikompaa: näytteistä suuri osa on vanhaa ja ne ovat yksikielisiä. Tietojenkalastelun kanssa ei kuitenkaan ole sekoitettu yleistä roskapostia. Järjestelmän etuna lisäksi mainittakoon, että sähköposteista muokattuja versioita tuottava vastapuoliagentti saattaa kompensoida heikkolaatuisia näytteitä, ja antaa lisäksi järjestelmälle paremmat edellytykset havaita laajoilla kielimalleilla tuotettua tietojenkalastelua, vaikka itse näytteiden valinnassa ei tätä ollutkaan huomioitu.

Esitettyjen menetelmien tarkastelun yhteenvetona voidaan todeta seuraavat asiat: (i) laajat kielimallit päihittivät kone- ja syväoppimisen vertailumenetelmät, (ii) usean agentin järjestelmä päihitti yksittäisen agentin järjestelmän, (iii) usean agentin järjestelmässä eri malleja yhdistelevät kokoonpanot pärjäsivät saman mallin kokoonpanoja paremmin, (iv) kehoteohjelmointitekniikoiden käytön vaikutus suorituskykyyn on epäselvä, (v) keskeisenä haasteena tutkimuksessa ovat testiaineistot, jotka eivät täysin vastaa todellisuutta ja (vi) laajoilla kielimalleilla tuotetun tietojenkalastelun uhkaa käsiteltiin tarkastelluissa tutkimuksissa vain rajallisesti.

Tulevaisuuden tutkimussuuntia pohtiessa korostuu erityisesti tarve testiaineistojen ajantasaistamiselle, ylläpidolle ja yhtenäistetylle käytölle. Mahdollisena ratkaisuna ovat talkoistetut (crowdsourced), jatkuvasti päivitetyt sähköpostiaineistot, kuten Koide et al. [22] käyttämä phishing_pot. Laajoilla kielimalleilla käytetyn tietojenkalastelun huomioimiseksi voitaisiin käyttää laajoilla kielimalleilla tuotettuja testiaineistoja, kuten ehdotettu Pajola et al. [35] toimesta. Lisäksi menetelmien rajoitteena ovat laajojen kielimallien laskennallisen vaativuuden myötä korkeat käyttökustannukset. Toimiva vaihtoehto olisi käyttää vanhempia laajoja kielimalleja, jotka tarjoavat kilpailukykyistä suorituskykyä huomattavasti alhaisemmin käyttökustannuksin, kuten Koide et al. [22] ehdottivat.

Contents

1 Introduction **1**

2 Background **3**

2.1 Email phishing attacks 3

2.2 LLM-generated phishing 4

2.3 LLMs 6

3 LLM-based methods for phishing email detection **9**

3.1 Proposed methods 10

3.1.1 ChatSpamDetector 10

3.1.2 Debate-driven multi-agent system 11

3.1.3 MultiPhishGuard 11

4 Analysis of LLM-based methods **14**

4.1 Results 14

4.2 Benchmark datasets 18

5 Conclusions **22**

Bibliography **26**

1 Introduction

Despite decades of research on countermeasures, phishing attacks (i.e., a victim being lured into disclosing sensitive information to an attacker posing as a trustworthy entity) continue to inflict monetary and reputational harm on individuals and organisations.

The technological simplicity of phishing makes it an easily accessible and highly popular form of cybercrime. According to Statista's 2024 U.S. cybercrime data [45], phishing was the most common cybercrime to affect individuals. Similarly, IBM's 2025 analysis covering 600 organisations affected by data breaches worldwide found phishing to be the most frequent attack vector, accounting for 16% of breaches, with an average cost of 4.8 million U.S. dollars per phishing-caused breach. In the second quarter of 2025, the Anti-Phishing Working Group (APWG) reported observing 1,130,393 phishing attacks globally, a count that has been steadily rising over the past year [4].

With the emergence and broad adoption of powerful large language model (LLM)-based tools, the domain of phishing is facing further change. This development presents both good and bad news: the exceptional abilities of LLMs, including a deep understanding of context in language, along with text generation and reasoning capabilities, make them highly effective and adaptable for both generating and detecting phishing attacks.

With LLM-generated phishing attacks, the economics of phishing are affected: if phishing campaigns require less skill, time, and effort to launch, their associated costs are lowered, presenting greater incentives for attacks [19], potentially leading to an increase in phishing attacks. Furthermore, the performance of various currently used phishing detection methods has been found to decrease with LLM-generated phishing emails compared to traditional ones [1]. With both the quantity and quality of phishing attacks being driven up at the same time, a significant challenge is presented.

Fortunately, LLM-based methods also show potential in addressing the challenges of the currently used phishing detection methods. Recent studies suggest that such methods can outperform classical machine learning (ML) and deep learning (DL)-based methods in phishing detection [22, 55], improve explainability by providing rationale for phishing email classifications [32, 22]. In addition, LLM-based approaches have been proposed to alleviate training data scarcity and support the detection of LLM-generated phishing through dataset augmentation [9] and fully synthetic LLM-generated datasets [35].

However, concerns have been raised regarding the evaluation of detection methods using benchmark datasets that do not adequately reflect present-day phishing attacks [35]. Given these concerns and the highly impressive results reported in research, we seek to examine and critically analyse prior work, with a primary emphasis on the benchmark datasets used.

Motivated by: (i) phishing remaining an open problem despite extensive research, (ii) the exceptional capabilities of LLMs in natural language processing (NLP) tasks, and (iii) the emerging threat of LLM-generated phishing, this thesis explores and analyses prior work on LLM-based methods for phishing attack detection. Specifically, we limit our scope to email, a primary medium for phishing attacks; other media, such as SMS, are not considered. We aim to understand the state of the art, identify potential challenges and limitations, and outline directions for future research.

The rest of the thesis is organised as follows: Section 2 provides context and background information on phishing email attacks and LLMs. Section 3 presents LLM-based methods for phishing email detection. Section 4 provides an analysis of the presented methods. Finally, Section 5 concludes the paper.

2 Background

This section provides relevant context and information on email phishing attacks (2.1), LLM-generated phishing (2.2), and technical background on LLMs (2.3).

2.1 Email phishing attacks

Phishing involves an attacker luring a victim into disclosing sensitive information (e.g., login, personal, financial), typically by the victim submitting the information on a phishing website (often a fake login page) or opening a malicious file attachment (e.g., a fake invoice) [23]. Attackers are often motivated by financial gain and exploit the compromised information for criminal purposes, such as identity theft, financial fraud, extortion, or espionage, causing victims financial and reputational harm [17, 20].

To appear trustworthy, attackers typically impersonate an entity (person or organization) that the victim knows and trusts, such as a co-worker, authority figure (e.g., a higher-up or law enforcement), or service provider they use (e.g., a bank) [23], with the three most commonly impersonated entities being Microsoft, Docusign, and organizations' human resources departments [5].

This facade is reinforced with methods of technical deception, such as imitating the email style of the impersonated entity (i.e., clone phishing), sender information forgery (i.e., spoofing), and using domain names that resemble those of the impersonated entity (e.g., microsoft.com vs. microsoftt.com) [16]. An example of a Microsoft Office clone phishing email is depicted in Figure 2.1.

Phishing messages typically use language that is urgent and threatening in nature, for example, prompting victims to resolve account issues, sign urgent documents, or respond to attention-grabbing human resources matters such as layoffs or salary changes [5].

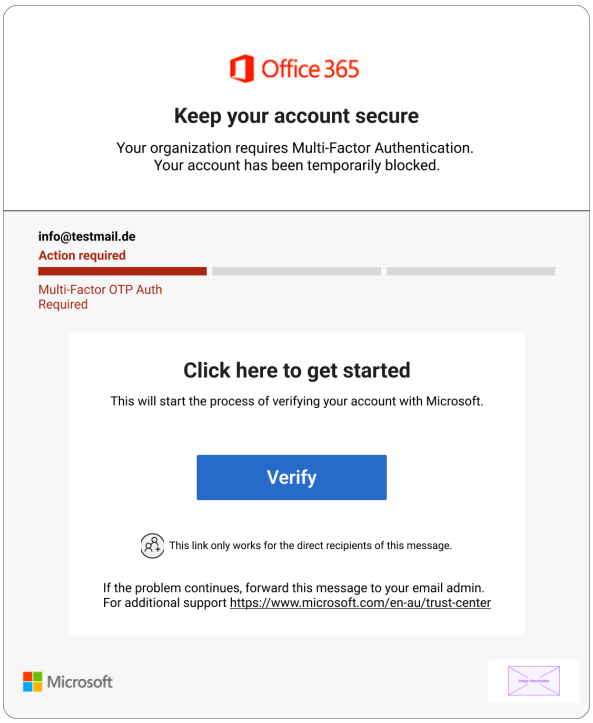

Figure 2.1: An example of a Microsoft Office phishing email [53].

2.2 LLM-generated phishing

The commercially available, vanilla (i.e., unmodified) LLMs ChatGPT [7, 33], Claude [2], and Bard [37] were found to be susceptible to misuse by phishing attackers, with Roy et al. [40] reporting that the models could be used to generate highly tailored and sophisticated phishing email attacks and websites. While the models refused to answer clearly malicious prompts, they could be tricked by dividing the malicious task into benign-seeming subtasks. For instance, the models could generate components for a phishing website which could be seamlessly combined to form a functional end product, as depicted in Figure 2.2. In addition, the models could produce further malicious prompts of their own that successfully evaded safeguards.

Although such vulnerabilities are regularly disclosed to model developers and subsequently patched, model safeguards remain imperfect. With evasion techniques (e.g., dividing a clearly malicious task into benign-seeming subtasks) exploiting the gray area between legitimate and malicious use, the issue is challenging to address through model safeguards alone. For example, the use of overly strict safeguards could prove impractical by causing collateral damage to legitimate users as a result of blocking harmless prompts. Moreover, attackers can simply bypass safeguards entirely by using

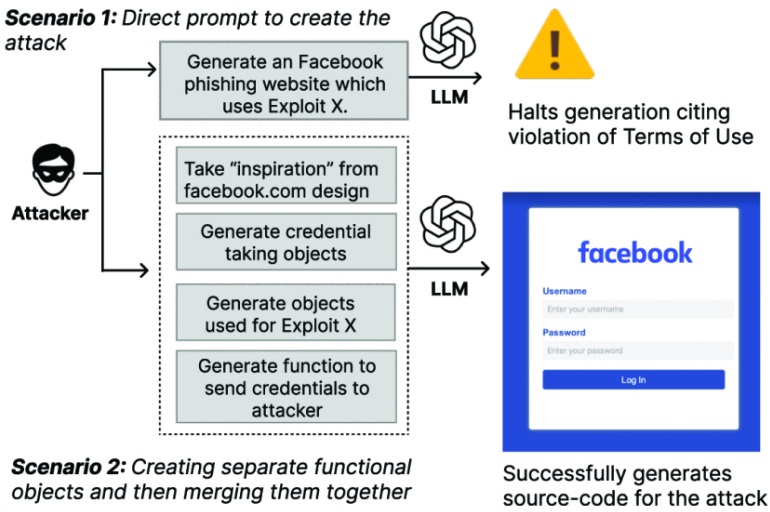

Figure 2.2: Tricking an LLM to generate code for a phishing website [40].

locally run modified open-source LLMs. As a result, prevention alone is not enough to address the problem, and the detection of LLM-generated phishing remains a relevant problem.

Given these factors, it is clear that LLM-generated phishing is here to stay. This raises the question of how prominent it is and how it is being used in practice. According to a recent phishing trends report by Hoxhunt [5], 0.7-4.7% of phishing emails (n=386,000) received by enterprise customers of Hoxhunt in 2024 were estimated to be AI-generated. This suggests that LLM-generated phishing is already a reality, though still an emerging phenomenon.

Specifically, a rise in the use of LLMs to generate tailored spearphishing attacks (i.e., targeting a specific person of interest) was reported, suggesting that while the number of LLM-generated phishing attacks is still low, they are highly sophisticated and tailored in nature. Indeed, attackers might find LLMs particularly useful for efficiently tailoring attacks to fit any individual's profile (e.g., gathered from online sources such as LinkedIn) [35].

2.3 LLMs

Large language models (LLMs) are neural language models that generate text by predicting the next token (i.e., a discrete unit of text that may correspond to a word, subword, character, or punctuation) in a sequence, given a preceding context of tokens [21]. They are pre-trained on vast amounts of text, and can predict tokens given a context of tens of thousands of tokens.

Modern LLMs, such as GPT-3 (Generative Pre-trained Transformer) [7], are typically autoregressive (i.e., the model predicts the next token given the previous token*) and are based on the transformer architecture [50], a neural network architecture based on the attention mechanism, introduced in 2017 by Vaswani et al. The predictions made by LLMs are shaped by their parameters (or weights) that are adjusted during pretraining and sometimes further fine-tuned through techniques such as reinforcement learning from human feedback (RLHF) [21].

For an LLM to process raw input text, the text sequence is segmented into a sequence of tokens in a process called tokenization. A commonly used tokenization method (or tokenizer) is byte-pair encoding (BPE) [43], which converts text to tokens based on a learned vocabulary and outputs a sequence of token IDs [21].

Each token is then converted to an embedding, a learned vector representation that encodes semantic information about the token in high-dimensional space. In this space, tokens with similar meanings often have similar vector representations [21, 42]. Because token embeddings do not contain positional information about the tokens, transformers incorporate positional embeddings to encode the positions of tokens within the sequence [50].

The core operation of the transformer is the self-attention mechanism, which incorporates contextual information from across the input sequence to produce contextualised representations of the input embeddings [21]. For a given sequence of input embeddings, each embedding is linearly projected into a query, a key, and a value vector q, k, and v. These are obtained by multiplying the input embeddings with learned weight matrices W^Q, W^K, and W^V [50, 21]. Self-attention then computes a weighted sum of the value vectors, where the weights are determined by dot product compatibility scores between the corresponding query and key vectors. Higher compatibility

*Other approaches include bidirectional language models such as BERT [15], a masked language model that predicts a token given the tokens both to the left and the right [21].

scores indicate that information from one token should more strongly contribute to the contextualised representation of another token [42].

In practice, the query, key, and value vectors are grouped into the matrices Q, K, and V to allow attention to be simultaneously computed over the entire input sequence. The scaled dot product attention mechanism is defined by Vaswani et al. [50] as:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

where the dot products are divided by the square root of the query and key vectors' dimensionality d_k, which is done in order to prevent attention scores from growing excessively large and push the softmax function into regions with extremely small gradients [50, 21]. The softmax function converts the attention scores into a probability distribution over the tokens, where larger attention weights indicate stronger contextual relevance between two tokens, while smaller values indicate weaker relevance [21].

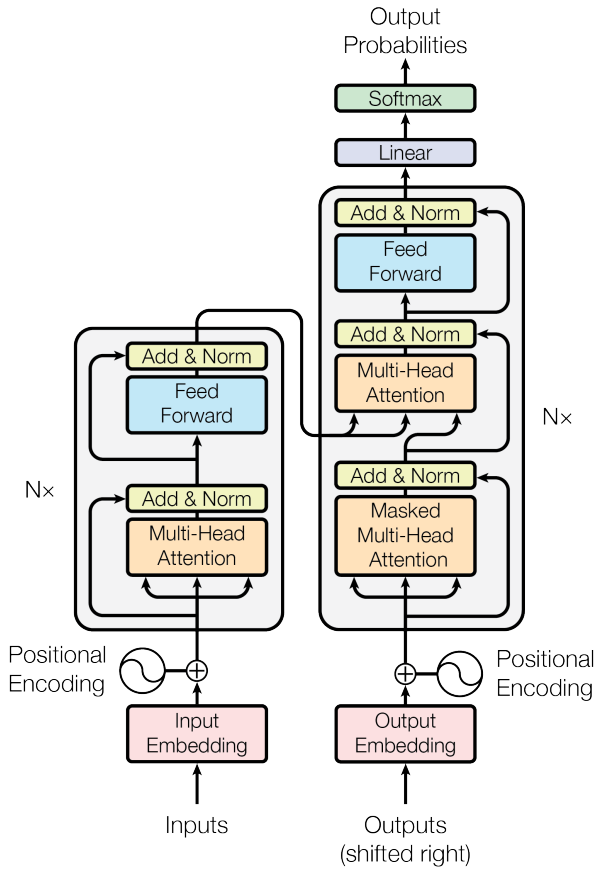

Figure 2.3: The transformer architecture with an encoder layer on the left and a decoder layer on the right. [50].

Rather than computing a single attention operation, transformers use multi-head at-

tention, in which multiple attention heads run in parallel [50, 21]. With h attention heads, each head i has its own set of learned weight matrices W_i^Q, W_i^K, and W_i^V. This allows the model to linearly project the input embeddings into different representation subspaces and attend to different types of relationships within the input sequence simultaneously. The outputs of the individual attention heads are then concatenated and linearly projected by an output weight matrix W^O to produce the final attention output.

At a high level, the transformer architecture consists of an encoder and a decoder stack [50], as depicted in Figure 2.3. The encoder stack consists of multiple identical layers that each contain two sublayers: a multi-head attention mechanism followed by a position-wise feed-forward network. Residual connections followed by layer normalization are applied around each sublayer.

The decoder stack follows a similar structure, but includes a third encoder-decoder multi-head attention sublayer in each layer, which allows the decoder to attend to the outputs of the encoder [50]. In addition, a masked multi-head attention mechanism is incorporated in the decoder to prevent tokens from attending to future positions. This masking preserves the model's autoregressive property by preventing information leakage and ensuring that each token prediction only depends on previously generated tokens [21].

3 LLM-based methods for phishing email detection

In this section, we present and summarize current research on LLM-based methods for phishing email detection. We begin by explaining the task of phishing email detection and key characteristics of LLM-based phishing email detection methods, after which we examine three recent methods proposed by Koide et al. [22], Nguyen et al. [51], and Xue et al. [55].

Phishing email detection is a binary classification task, in which a detector model classifies a given email as phishing or legitimate. It is especially critical to minimize the number of false negative misclassifications (i.e., a phishing email is incorrectly classified as legitimate). Though false positives (i.e., a legitimate email is classified as phishing) present an annoyance to users and are desirable to minimize, they are considered to be secondary in priority. As explained in the previous section, the capabilities of LLMs (e.g., being able to capture deeper context and nuance in natural language, being trained on vast amounts of information, and having a large context size) offer great promise for accurately classifying emails.

With classical ML [26, 6] and DL [30, 13] methods only returning a classification label for a given email, users might be left confused as to why an email was classified in a certain way. LLM-based methods address this by returning a more detailed classification report that, in addition to the classification label itself, also contains the decision rationale [51], along with some methods returning a confidence score (i.e., how confident the model is in the decision) [22, 55] with the aim of helping users understand and better evaluate the nature of received emails.

In addition, the use of prompt engineering (i.e., defining specific instructions for models) is often explored with LLM-based solutions [54], whether to customize model outputs (e.g., defining a certain response structure for consistency) or to seek performance improvements via techniques such as chain-of-thought reasoning (i.e., instructing the model to divide a task into several steps) and role prompting (i.e., instructing the model to adopt a role, such as a cybersecurity analyst).

3.1 Proposed methods

3.1.1 ChatSpamDetector

Koide et al. propose ChatSpamDetector [22], a single-agent LLM system. The system architecture is detailed in Figure 3.1. The use of four commercially available LLMs is evaluated: GPT-3.5-Turbo [7], GPT-4 [33], Llama 2-70B [48], and Gemini 1.0 Pro [46]. Two prompt versions, a normal and a simple variant, are used. The normal prompt makes use of the previously described prompt engineering techniques: the model is instructed to adopt the role of a spam detector (role prompting) and divide email analysis into five subtasks (chain-of-thought reasoning).

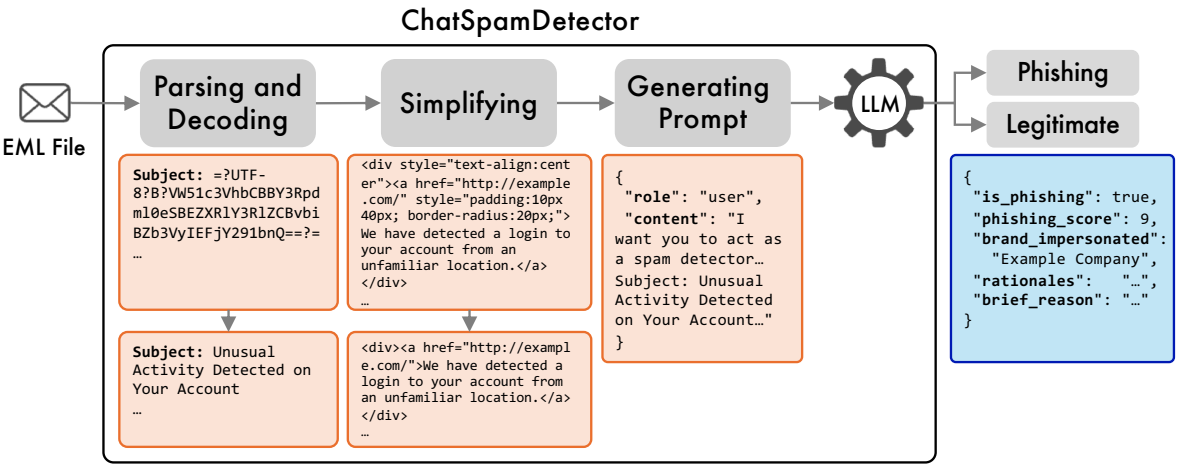

Figure 3.1: Architecture of the ChatSpamDetector system [22].

The system processes each email by parsing and decoding its contents and, if necessary, simplifying the data: if the email's token count exceeds a limit of 3,000 (based on the 4,000 token limit of Llama 2), content is progressively omitted following a rule-based simplification process [22]. For multipart emails containing multiple data types (e.g., HTML and plaintext versions), the system keeps the HTML version while removing other formats' versions*. The system then removes nonessential HTML style and metadata elements while preserving elements commonly used in phishing attacks, such as hyperlinks and their associated anchor text. If the token limit is still exceeded after these steps, HTML elements are removed from the center of the message until the input

*This is based on Koide et al.'s [22] observation that phishing emails commonly utilize HTML-specific tactics, such as deceptive hyperlinks that display anchor text that differs from the underlying URL.

falls within the allowed length. These steps are based on the assumptions that phishing emails primarily aim to direct victims to click on links and that phishing-related information is more likely to appear near the beginning or end of an email.

The system then generates a prompt containing the processed email data and provides it to an LLM agent for analysis. The LLM agent produces a structured report containing a binary classification label, a confidence score (from 0 to 10), the impersonated brand (if applicable), a detailed rationale (up to 500 words), and a brief reasoning for the classification [22].

3.1.2 Debate-driven multi-agent system

In addition to single-agent methods, the use of multi-agent methods has been explored [51, 55]. LLM-based methods may face the problem of confirmation bias, meaning that models might tend to avoid investigating other possible paths of interpretation after having formed an initial reasoning path. This can be problematic in cases such as phishing emails that look legitimate at first glance, as is often the case, requiring examination from different perspectives for sound evaluation.

To address this, Nguyen et al. propose a multi-agent debate-driven system [51], as depicted in Figure 3.2. The use of two different LLMs is evaluated: GPT-4 [33] and Llama 2 [48]. The system uses three LLM agents with pre-defined roles. Two agents engage in two rounds of debate regarding the classification label of the given email, with the first agent arguing for the email being phishing, and the second arguing for the email being legitimate. After the debate process is finished, the third agent acting as a judge reviews the presented arguments and provides the final classification label. The system outputs a binary classification label and the decision rationale of the judge agent. As with Koide et al., Nguyen et al. also use prompt engineering techniques: they evaluate the use of chain-of-thought reasoning and role prompting [51].

3.1.3 MultiPhishGuard

Xue et al. propose a five-agent system, MultiPhishGuard [55]. A single LLM, GPT-4o [34], is used for all five agents. From left to right in the system architecture depicted in Figure 3.3: an adversarial agent creates variation by producing rephrased variants of emails in the datasets used. This is an intriguing decision and could prove highly

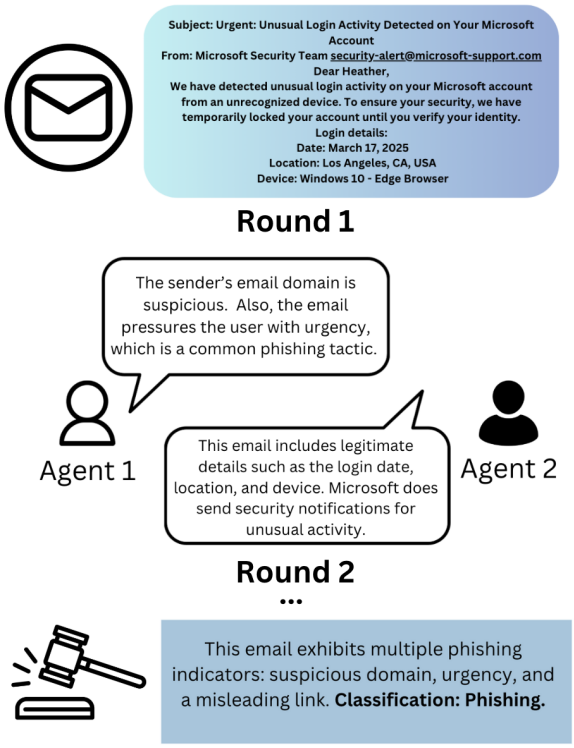

Figure 3.2: Multi-agent debate process, where two LLMs engage in two rounds of debate, after which a judge agent evaluates their arguments and gives a final classification decision. [51]

valuable in accounting for LLM-generated or -rephrased phishing. Three agents analyze the email content itself, with each focusing on a specific subsection (body, URLs, metadata), and returning their individual reports containing a classification label, a confidence score, and the decision rationale. The last agent produces a simplified summarization of the agent reports for easier-to-read conclusions.

Additionally, a reinforcement learning method, proximal policy optimization (PPO), is used for the system to dynamically adjust the analysis agent output weights (e.g., how important the output of any one analysis agent is to the final verdict) [55]. These weights are fine-tuned during the training period of the system. In testing, the optimized weights are used to weigh agent outputs depending on the extracted features of the analyzed email. Xue et al. reason that this dynamic weight adjustment mechanism ensures "that each email is evaluated according to its unique characteristics". Indeed, it might be valuable not to assign equal importance to all the analysis agents if, for example, the analyzed email contains legitimate-seeming metadata and text, but the embedded URLs are highly suspicious.

Finally, based on the weighted analysis agent verdicts and confidence scores, the sys-

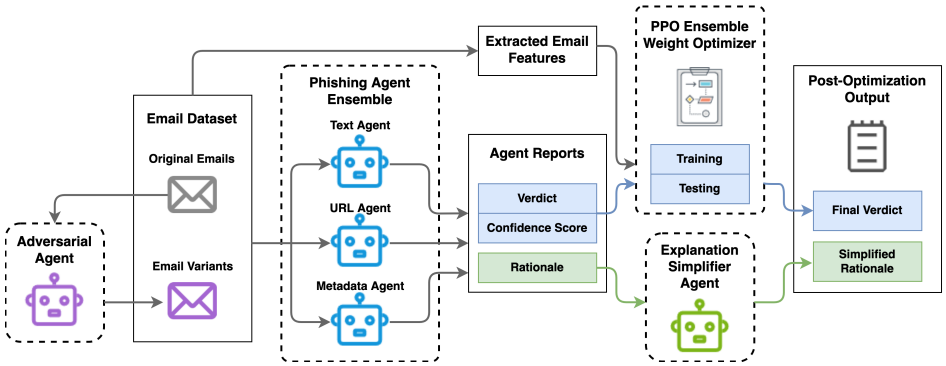

Figure 3.3: Architecture of MultiPhishGuard, consisting of five specialized LLM-based agents. From left to right, these are the: i) adversarial agent, ii) text agent, iii) URL agent, iv) metadata agent, and v) explanation simplifier agent [55].

tem returns a final verdict and the simplified rationale produced by the explanation simplifier agent [55].

4 Analysis of LLM-based methods

This section presents an analysis of the LLM-based phishing email detection methods described in Section 3. We assess the reported performance of each method (4.1), and consider the benchmark datasets used (4.2).

4.1 Results

In evaluating phishing email detection methods, a key concern is false negative classifications (i.e., phishing emails that are incorrectly classified as legitimate), as explained in Section 3.1. Minimizing false negatives is critical, as phishing emails that evade detection can directly lead to financial loss, data breaches, or identity theft. Phishing emails are inherently malicious in intent and aim to deceive recipients into revealing sensitive information, which makes phishing a high-stakes classification task.

In contrast, general spam detection can be viewed as a lower-stakes problem. While spam emails can occasionally also be malicious, they are more commonly more benign in intent (e.g., unsolicited or misleading advertisements for questionable products or services) [44]. Thus, the consequences of spam emails evading detection are typically less severe.

This difference in risk profiles means that evaluation approaches can differ between the two tasks: for spam detection, a balanced approach might be taken to reduce both false negatives and false positives (i.e., the number of legitimate emails incorrectly classified as phishing, which are typically more of an annoyance than harmful). In phishing detection, however, specific emphasis is commonly placed on minimizing false negatives, as even a small number of undetected phishing emails can inflict significant harm on users.

Given the risks associated with phishing emails evading detection, we place specific emphasis on metrics that reflect the correct classification of phishing emails. The first of these is recall, which measures the proportion of phishing emails correctly identified by the system:

$$\text{Recall} = \frac{TP}{TP + FN}$$

where TP (true positives) are phishing emails correctly classified as phishing, and FN (false negatives) are phishing emails incorrectly classified as legitimate. Recall reflects the system's ability to prevent phishing emails from evading detection, with higher values indicating more effective protection [55]

To explicitly quantify missed phishing detections, we also report the false negative rate (FNR):

$$\text{FNR} = \frac{FN}{TP + FN}$$

which represents the proportion of undetected phishing emails. A low FNR is particularly important, as even a small number of undetected phishing emails can cause significant harm to users [55].

As a supplementary measure, we report accuracy, which reflects overall classification performance across both phishing and legitimate emails:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where TN (true negatives) are legitimate emails correctly classified as legitimate, and FP (false positives) are legitimate emails incorrectly classified as phishing. While accuracy gives a broad summary of performance [55], it does not distinguish between error types and thus is less informative in the context of phishing detection, where false negatives are more critical compared to false positives.

The results reported by Koide et al. [22] and Xue et al. [55], as summarized in Table 4.1*, indicate that the proposed methods performed impressively in phishing email detection, achieving high recall, low false negative rate, and high accuracy. Nguyen et al. [51], however, only reported accuracy, leaving out the recall and false negative metrics that are of specific interest in evaluating phishing email detection performance. We note that the reported results are not directly comparable, as the methods were evaluated using different benchmark datasets. These will be further discussed in Section 4.2.

The evaluation results of Koide et al. [22] indicate that LLMs outperformed baseline machine learning and deep learning methods, which included a random forest (RF) [6], a support vector machine (SVM) [26], an artificial neural network (ANN) [30], and a convolutional neural network (CNN) [13]. We note, however, that out of the

*For clarity, the metrics reported in the table were either explicitly provided or could be calculated based on the reported results.

Method	LLM	Recall	FNR	Accuracy
Koide et al. [22]	GPT-4	99.70%	0.30%	99.70%
Nguyen et al. [51]	GPT-4-Llama-2-GPT-4	-	-	99.07%
Xue et al. [55]	5x GPT-4o	99.80%	0.20%	97.89%

Table 4.1: False negative rate (FNR), recall, and accuracy score for the best-performing configuration of each method. Note that the values are not suited for direct comparison due to the different benchmark datasets used. The FNR for Koide et al. was calculated based on reported FN and TP values. Blank slots for Nguyen et al. signify the values were not reported and could not be calculated based on the available information.

evaluated baseline methods, the RF and SVM were specifically trained for phishing email detection, while the ANN and CNN methods were trained for general-purpose spam email detection. Given that phishing and spam emails can be "semantically very different" [35], this may have set the ANN and CNN at a disadvantage compared to the other methods. As a result, their reported performance might not accurately portray the capabilities of ANNs and CNNs specifically trained for phishing detection.

Out of the evaluated LLMs, GPT-4 performed the best [22], achieving a recall of 99.70%, a false negative rate of 0.30%, and an accuracy of 99.70% when using the normal prompt. Using GPT-4 with the simple prompt yielded slightly worse results, with a recall of 99.11%, a false negative rate of 0.89%, and an accuracy of 99.35%. This indicates that incorporating prompt engineering techniques (chain-of-thought reasoning, role prompting, and information about social engineering techniques) in the normal prompt improved GPT-4's performance in classifying both phishing and legitimate emails.

For the other evaluated LLMs (GPT-3.5-Turbo, Llama 2-70B, and Gemini 1.0 Pro), however, using the normal prompt did not lead to an overall improvement in email classification performance; rather, it led to a reduction in false negatives but an increase in false positives. This indicates that the use of prompt engineering techniques made the models more aggressive, letting fewer phishing attacks evade detection at the cost of more false alarms (legitimate emails incorrectly classified as phishing). Given that it is critical to minimize the number of false negative classifications, this trade-off can be considered to be worth it. Overall, the results of Koide et al. suggest that prompt engineering techniques can improve phishing email detection performance.

As stated earlier, the results of Nguyen et al. [51] did not include recall or false negative rate, which are the metrics of specific importance for phishing detection. Their evaluation focuses on accuracy and F1-scores, which portray overall performance (i.e., classification performance across phishing and legitimate emails) instead of specifically focusing on missed phishing attacks, which represents a limitation of the evaluation in the context of phishing detection.

The reported results indicate that mixed-model configurations consistently outperformed same-model configurations, with the best-performing GPT-4-Llama-2-GPT-4 setup achieving a combined accuracy of 99.07% compared to 98.16% for the Llama-2-Llama-2-Llama-2 configuration [51]. While the difference is not drastic, these findings suggest that the use of mixed-agent configurations in multi-agent systems may offer benefits over same-agent configurations, with Nguyen et al. noting that mixed agents may complement one another's reasoning capabilities (e.g., by offsetting model-specific biases). However, because phishing-specific metrics were not reported, it remains unclear whether this would translate to improved phishing email detection performance, specifically.

In contrast to the findings of Koide et al. [22], Nguyen et al. [51] report that chain-of-thought reasoning and role prompting did not improve email classification performance either separately or in combination. The contrasting results may be explained by differences in system architectures (single-agent vs. multi-agent) or the debate-driven analysis process that Nguyen et al. used: given that email analysis is already structured as a two-stage reasoning process involving multiple agents and a final judging step, further chain-of-thought reasoning and role prompting techniques may offer limited benefits. Nevertheless, these results should, again, be interpreted with caution when drawing conclusions about phishing detection performance.

Lastly, the results of Xue et al. [55] indicate that the five-agent system outperformed the baseline methods, which included a single agent, a single agent with chain-of-thought reasoning (CoT), and a RoBERTa [25] (i.e., a fine-tuned BERT language model). The system achieved a recall of 99.80%, a false negative rate of 0.20%, and an accuracy of 97.89 %. With the baseline methods including a single agent with and without CoT, it can be observed that the single agent with CoT had a slightly lower recall of 99.08% and a slightly higher false negative rate of 0.92% compared to the single agent without CoT, which had a recall of 99.39% and a false negative rate of 0.61%, although accuracy saw a slight improvement with CoT. These results suggest

that the use of a single agent with CoT increased the number of missed phishing emails, resulting in a weaker phishing detection performance.

In summary, these results suggest that: (i) LLM-based approaches may outperform machine learning and deep learning methods in phishing email detection, (ii) multi-agent systems potentially offer benefits over single-agent systems, (iii) within multi-agent systems, mixed-model configurations might outperform same-model configurations in general email classification, and (iv) the impact of chain-of-thought reasoning and role prompting techniques on phishing detection performance remains unclear.

4.2 Benchmark datasets

In this section, we evaluate the datasets used to benchmark model performance. We identify several limitations in currently widely used datasets, including outdated samples, monolinguality, and the inclusion of general spam instead of a strict focus on phishing. We also investigate whether the emerging threat of LLM-generated phishing, discussed in Section 2.2, is being accounted for in the data used.

The performance of a detector model is evaluated using benchmark datasets containing phishing and benign emails. For a model's evaluated performance to accurately translate into real-world performance, the chosen benchmark dataset should comprehensively reflect the current landscape of phishing attacks, that is, consist of recently collected email samples in multiple linguistic and cultural contexts. To account for the rise in LLM-generated emails, both phishing and legitimate, the inclusion of LLM-generated email samples could be beneficial.

The benchmark dataset used by Koide et al. [22], as summarized in Table 4.2, consisted of recently collected, multilingual email samples. No general spam was mixed in with phishing email samples, meaning that the benchmark data is in line with the intended scope of phishing detection. Phishing email samples were sourced from phishing_pot [36], an actively updated, community-maintained, and multilingual phishing sample repository. As phishing_pot solely contains phishing, legitimate emails were sourced from the CSDMC 2021 [12] public datasets.

Overall, this dataset reflects current-day phishing reasonably well and can be considered to be of comparatively high quality. As a small caveat, no LLM-generated phishing

samples were included in the benchmark data*. To ensure coverage of LLM-generated phishing attacks, one option would be to generate LLM-rephrased variants of samples in the dataset, as is done by the adversarial agent implemented in the system proposed by Xue et al. [55], or use LLM-generated samples [35].

Source	Collected	Phishing	Legitimate	Language
phishing_pot repository [36]	2022-2023	1010	0	19 languages
CDMC spam corpus [12]	2021	0	1000	12 languages
Total		1010	1000	

Table 4.2: Summary of the email benchmark data used by Koide et al. [22] to evaluate phishing email detection performance. The data is of high overall quality, consisting of recently collected samples spanning multiple languages, with no generic spam mixed in with phishing samples. A small deficiency is the lack of LLM-generated phishing samples.

The benchmark dataset used by Nguyen et al. [51], summarized in Table 4.3, also contained a balanced mix of phishing and legitimate email samples. However, most email samples are very outdated, being collected in the late 1990s and early 2000s, though some emails from the Nazario-5 [8] might have been collected as recently as 2022. Only English email samples are covered, and some general spam seems to have been mixed in with phishing samples, given the inclusion of malicious emails from the SpamAssassin public corpus [38] and the Ling-Spam dataset [41].

The use of the UoT [29] phishing dataset is an intriguing choice: released in 2024, it consists of real and "artificially generated" synthetic samples. The time of collection and generation of the samples has not been specified, nor the generation method itself. It is possible that the real samples have been recently collected and the synthetic samples generated using LLMs, which would make them a sound addition to the benchmark data. However, the lack of concrete details makes it difficult to assess the quality of the data.

Overall, the benchmark dataset seems to consist of outdated and monolingual samples with general spam potentially mixed with phishing. As a result, the models were evaluated on test data unrepresentative of modern-day phishing, and the benchmark data might not have been fully in line with the intended scope of phishing detection.

The benchmark dataset used by Xue et al. [55], summarized in Table 4.4, also seems to

*Though a small number of LLM-produced emails may be present in the 2022-2023 email samples.

Source	Collected	No. phishing	No. legitimate	Language
Nazario-5 dataset [8]	2005-2022	1565	1500	English
CLAIR collection of fraud email [39]	1998-2007	3332	0	English
SpamAssassin public corpus [38]	2002-2006	1718 (part spam)	4091	English
UoT phishing dataset [29]	Not specified	1000	1000	English
Ling-Spam dataset [41]	2000	458 (part spam)	2401	English
Total		8073	8992	
After omission of long emails		6323	6475	

Table 4.3: Summary of benchmark emails used by Nguyen et al. [51]. Emails that could exceed LLM token limits were omitted from the test dataset. The Nazario-5 dataset [8] is a variant of the Nazario [31] phishing corpus combined with legitimate emails from TREC 2005-2007 [49] and CEAS 2008 [10] public corpora. The UoT [29] phishing dataset consists of real and "artificially generated" samples: the time of collection and generation of the samples was not specified, nor the generation method.

contain samples of varied quality. Except for the 402 emails sourced from the Nazario phishing corpus [31] being collected in 2024, the samples are very outdated (1998-2008) and entirely monolingual (English). As a positive aspect, no general spam was mixed in with phishing email samples, meaning that the benchmark data focuses specifically on phishing detection, albeit by using primarily outdated samples.

No LLM-generated phishing samples were included in the benchmark data, though the inclusion of an adversarial LLM agent that generates variations of email samples from the dataset may have at least partially addressed this deficiency.

Source	Collected	No. phishing	No. legitimate	Language
Nazario phishing corpus [31]	2024	402	0	English
CLAIR collection of fraud email [39]	2007	577	0	English
SpamAssassin public corpus [3]	2004	0	500	English
Enron-Spam dataset [28]	1999-2002	0	1500	English
TREC public corpus [11]	2007	0	500	English
CEAS public corpus [10]	2008	0	500	English
Total		979	3000	

Table 4.4: Summary of benchmark emails used by Xue et al. [55]. The data is more tilted towards legitimate samples at an approximate 1:3 ratio. Except for 402 phishing samples from 2024, the samples are very outdated (1999-2008), and entirely monolingual (English). The authors ensured a pure focus on phishing email detection by only including legitimate email samples from datasets that mix phishing with general spam.

In summary, an examination of the proposed methods' benchmark data reveals several issues, including the use of outdated data, general spam being mixed with phishing samples, and monolinguality, with datasets typically only containing emails written in English. In addition, LLM-generated phishing was fully or almost fully unaccounted for in the benchmark datasets, with the only potential exception being the UoT phishing dataset [29], which contained a mix of real and synthetic email samples, though the method used to generate the samples was not reported.

Indeed, these findings are in line with the conclusions of Pajola et al. [35] who, in reviewing prior work on phishing email detection methods, identified an open problem widely present in current research: most work relies on just 8 datasets that share certain critical limitations: (i) almost all are outdated, containing emails collected before 2010, (ii) some mix phishing with general spam, leading to phishing detectors unintendedly being benchmarked in spam detection instead of specializing in phishing, and (iii) all are monolingual, only containing emails in English.

Outdated datasets pose a challenge to phishing detection research, as they do not reflect current-day trends in phishing, such as modern brand impersonation, cryptocurrency scams [22], or LLM-generated phishing emails that can be characterized by fluent language and being tailored to fit specific services [5] or user contexts [35]. In addition, older datasets can reflect outdated phishing trends, such as the advance-fee scams in the CLAIR dataset [39], which rely on stereotypical narratives (e.g., a foreign prince offering riches in broken English), further limiting their relevance. With phishing tactics evolving in response to continuous changes in online services and how users behave, evaluating models on older data may overestimate real-world performance and, as a result, fail to generalize to current phishing trends.

Monolingual benchmark datasets further limit the applicability of evaluation results, with real-world phishing campaigns becoming increasingly multilingual and targeting users across different linguistic and cultural contexts (e.g., the recent S-Bank phishing campaigns in Finland [52]). Models evaluated solely on English-language data might not generalize well to other languages and cultural contexts. As a result, the evaluation of models on monolingual datasets might risk underestimating the challenges involved with deploying phishing detection systems in globally used email services.

5 Conclusions

This section concludes the paper with a discussion on the presented solutions' impact, challenges, and limitations, along with recommendations and potential future research directions.

Due to the exceptional ability of LLMs to capture deep context and nuance in natural language, they show considerable potential for phishing email detection. Indeed, evidence suggests that LLM-based methods are able to outperform baseline machine and deep learning methods in the detection of both human-generated phishing [22] and LLM-generated phishing [35], demonstrating excellent adaptability for general phishing detection. LLM-based detectors also have other distinct advantages, including the ability to provide decision rationale for classifications.

However, we identify several challenges in current research: the use of benchmark datasets that are unrepresentative of reality (e.g., outdated, monolingual, general spam mixed with phishing samples), the threat of LLM-generated phishing attacks not being accounted for in benchmark data selection, and the higher associated costs of using LLM-based detection methods.

The use of outdated and otherwise limited datasets can partially be attributed to privacy concerns: individuals or organizations may not wish to release their private emails, resulting in a lack of publicly available email data. This consequently leaves researchers with limited options regarding dataset selection, leading to the persistent use of datasets that are readily available but of low quality.

Given the scarcity of real-world email samples and the lack of LLM-generated email samples in benchmark datasets, a promising (and privacy-friendly) course of action would be to address both concerns at once by investigating the use of LLM-generated benchmark datasets. For this purpose, Pajola et al. [35] propose the E-PhishGen framework to generate novel phishing email datasets (containing both phishing and legitimate emails) using LLMs. In evaluating selected machine learning-based detection methods on the LLM-generated benchmark dataset, they observe a decline in detector model performance, meaning that the LLM-generated data are much harder to classify compared to existing benchmark datasets. They also find that LLMs fare better in classifying the LLM-generated samples compared to the machine learning methods,

suggesting that the phenomenon of LLMs outperforming prior methods in phishing detection translates from outdated datasets to the detection of sophisticated LLM-generated phishing as well.

For sourcing real-world samples to complement LLM-generated data, one potential option is the use of crowdsourced, dynamically updated datasets. One such example is the phishing_pot repository [36] utilized by Koide et al. [22]. Community-maintained datasets have several advantages over any dataset maintained by a single author: a common issue with existing benchmark datasets is that they are published and then completely forgotten about, or updated for some time until the authors lose interest. While some exceptions exist, such as the Nazario corpus [31] maintained by its sole author, Jose Nazario, from 2005 to the present day, it still only depicts self-reported samples from a single person's inbox: a crowdsourced dataset would offer a more comprehensive picture; furthermore, it does not depend on the continued effort and motivation of any one author.

The compute and resource costs associated with developing and using state-of-the-art LLMs are significant, meaning that researchers are forced to rely on commercial LLM service providers. While this means there is no upfront investment, recurring API call fees could prove problematic when using a phishing detector that continuously processes incoming email. In analysing 2,010 emails with GPT-4 and using the normal prompt, Koide et al. [22] reported that the ChatGPT API calls incurred a total cost of 265.80 U.S. dollars, a high cost. Given the volume of incoming email that a large organization continuously processes, the associated costs might present a critical barrier for the adoption of LLM-based detection methods. However, Koide et al. found that using GPT-3.5 with a normal prompt incurred a cost of 13.68 U.S. dollars for the same amount of emails analysed, a greatly reduced cost. Given that the phishing detection performance of GPT-3.5 with the normal prompt was still very high (96.92% accuracy), a practical option could be using older models that are inexpensive to use, yet offer competitive performance.

Alternatively, given their lower operational costs, the local use of lightweight, open-source LLMs, such as DeepSeek R1 70B [14], and Llama 3.3 70B [27], could be explored. This has already been explored for phishing website detection [18], with the results of Goldenits et al. indicating that these lightweight, open-source LLMs (called small language models or SLMs by the authors) underperformed compared to state-of-the-art LLMs. In addition, their usage involves a significant upfront cost associated with the

initial hardware investment (e.g., an NVIDIA H100 series GPU costs 27,000-39,000 EUR). The authors note, however, that locally-run SLMs become economically viable under sustained workloads, as the upfront hardware investment costs are eventually offset by the recurring API costs of commercial alternatives. There may also be additional benefits beyond just the economic viability: for entities wanting to limit reliance on commercial providers and/or are concerned with the privacy implications of sending email data to the external systems of third parties, locally-run SLM-based phishing email detection systems could present an attractive solution.

Use of AI in the Thesis

I used Keenious and Scopus AI in sourcing relevant sources; however, I did not end up using the provided sources. I used ChatGPT 5.0, ChatGPT 5.2, and Grammarly Free tier for spellchecking.

For producing the extended Finnish abstract, I translated terms for which I found no established translations by using ChatGPT 5.1 and ChatGPT 5.2.

Bibliography

[1] K. Afane, W. Wei, Y. Mao, J. Farooq, and J. Chen. "Next-Generation Phishing: How LLM Agents Empower Cyber Attackers". In: *2024 IEEE International Conference on Big Data (BigData)*. 2024, pp. 2558–2567. DOI: 10.1109/BigData62323.2024.10825018.

[2] Anthropic. *Introducing Claude*. Mar. 2023. URL: https://www.anthropic.com/news/introducing-claude.

[3] *Apache SpamAssassin*. URL: https://spamassassin.apache.org/.

[4] APWG. *Phishing Attack Trends Report – 2Q 2025*. 2025. URL: https://docs.apwg.org/reports/apwg_trends_report_q2_2025.pdf (visited on 09/30/2025).

[5] E. Baker and M. Cartier. 2025. URL: https://hoxhunt.com/guide/phishing-trends-report.

[6] C. Beaman and H. Isah. *Anomaly Detection in Emails using Machine Learning and Header Information*. 2022. arXiv: 2203.10408 [cs.CR]. URL: https://arxiv.org/abs/2203.10408.

[7] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin. Vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

[8] A. I. Champa, M. F. Rabbi, and M. F. Zibran. "Curated Datasets and Feature Analysis for Phishing Email Detection with Machine Learning". In: *2024 IEEE 3rd International Conference on Computing and Machine Intelligence (ICMI)*. 2024, pp. 1–7. DOI: 10.1109/ICMI60790.2024.10585821.

[9] F. Chen, T. Wu, V. Nguyen, S. Wang, A. Abuadbba, and C. Rudolph. *PEEK: Phishing Evolution Framework for Phishing Generation and Evolving Pattern Analysis using Large Language Models*. 2025. arXiv: 2411.11389 [cs.CR]. URL: https://arxiv.org/abs/2411.11389.

[10] G. V. Cormack. *CEAS 2008 Public Corpus*. 2008. URL: https://plg.uwaterloo.ca/~gvcormac/ceascorpus/.

[11] G. V. Cormack and T. R. Lynam. *TREC 2007 Public Corpus*. 2007. URL: http://plg.uwaterloo.ca/~gvcormac/treccorpus07/.

[12] CSDMC. *CDMC 2021 datasets*. The 12th International Cybersecurity Data Mining Competition (CDMC 2021). July 2021. URL: https://csmining.org/cdmc2021/datasets/.

[13] Dbsheta. *DBSHETA/SPAM-detection-using-deep-learning: Detecting spam emails using CNN*. 2017. URL: https://github.com/dbsheta/spam-detection-using-deep-learning.

[14] DeepSeek-AI et al. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025. arXiv: 2501.12948 [cs.CL]. URL: https://arxiv.org/abs/2501.12948.

[15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by J. Burstein, C. Doran, and T. Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. URL: https://aclanthology.org/N19-1423/.

[16] Fortinet. *What is URL phishing?* URL: https://www.fortinet.com/resources/cyberglossary/url-phishing.

[17] *Fraud - Police*. Police of Finland. URL: https://poliisi.fi/en/fraud.

[18] G. Goldenits, P. Koenig, S. Raubitzek, and A. Ekelhart. *Small Language Models for Phishing Website Detection: Cost, Performance, and Privacy Trade-Offs*. 2025. arXiv: 2511.15434 [cs.CR]. URL: https://arxiv.org/abs/2511.15434.

[19] F. Heiding, B. Schneier, A. Vishwanath, J. Bernstein, and P. S. Park. "Devising and Detecting Phishing Emails Using Large Language Models". In: *IEEE Access*, 12 (2024), pp. 42131–42146. DOI: 10.1109/ACCESS.2024.3375882.

[20] IBM and P. Institute. 2025. URL: https://www.ibm.com/reports/data-breach.

[21] D. Jurafsky and J. H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd. Online manuscript released August 24, 2025. 2025. URL: https://web.stanford.edu/~jurafsky/slp3/ed3book_aug25.pdf.

[22] T. Koide, N. Fukushi, H. Nakano, and D. Chiba. *ChatSpamDetector: Leveraging Large Language Models for Effective Phishing Email Detection*. Accepted at SecureComm 2024. 2024. arXiv: 2402.18093 [cs.CR]. URL: https://arxiv.org/abs/2402.18093.

[23] M. Kosinski. *What is phishing?* URL: https://www.ibm.com/think/topics/phishing#732739699 (visited on 10/06/2025).

[24] N. Kumaran. *Understanding Gmail's spam filters*. May 2022. URL: https://workspace.google.com/blog/identity-and-security/an-overview-of-gmails-spam-filters.

[25] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. 2019. arXiv: 1907.11692 [cs.CL]. URL: https://arxiv.org/abs/1907.11692.

[26] Mo-Messidi. *Mo-Messidi/email-phishing-attempts-detection-using-NLP: Email phishing attempts detection from the text of email bodies using natural language processing and machine learning*. 2019. URL: https://github.com/mo-messidi/Email-Phishing-Attempts-Detection-using-NLP.

[27] Meta-Llama. *Llama 3.3 model card*. Dec. 2024. URL: https://github.com/meta-llama/llama-models/blob/main/models/llama3_3/MODEL_CARD.md.

[28] V. Metsis, I. Androutsopoulos, and G. Paliouras. "Spam Filtering with Naive Bayes - Which Naive Bayes?" In: *International Conference on Email and Anti-Spam*. 2006. URL: https://api.semanticscholar.org/CorpusID:18749628.

[29] R. Miltchev, R. Dimitar, and G. Evgeni. *Phishing validation emails dataset*. Feb. 2025. URL: https://zenodo.org/records/13474746.

[30] MoAbd. *MoAbd/SPAM-detection: Implementation of ANN for email spam-detection using TensorFlow*. 2016. URL: https://github.com/MoAbd/Spam-detection.

[31] J. Nazario. *Nazario Phishing Corpus*. 2024. URL: https://monkey.org/~jose/phishing/.

[32] V. Nguyen, T. Wu, X. YUAN, M. Grobler, S. Nepal, and C. Rudolph. "AI2TALE: An Innovative Information Theory-based Approach for Learning to Localize Phishing Attacks". In: *International Conference on Representation Learning*. Ed. by Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu. Vol. 2025. 2025, pp. 23303–23326. URL: https://proceedings.iclr.cc/paper_files/paper/2025/file/3a819a53408e20b75d1954bf617ccc0a-Paper-Conference.pdf.

[33] OpenAI et al. *GPT-4 Technical Report*. 2024. arXiv: 2303.08774 [cs.CL]. URL: https://arxiv.org/abs/2303.08774.

[34] OpenAI et al. *GPT-4o System Card*. 2024. arXiv: 2410.21276 [cs.CL]. URL: https://arxiv.org/abs/2410.21276.

[35] L. Pajola, E. Caripoti, S. Banzer, S. Pizzi, M. Conti, and G. Apruzzese. "E-PhishGen: Unlocking Novel Research in Phishing Email Detection". In: *arXiv preprint arXiv:2509.01791* (2025). Accepted to ACM AISec '25 [Best Paper Award]. URL: https://arxiv.org/abs/2509.01791v2.

[36] rf-peixoto. *RF-Peixoto/phishing_pot: A collection of phishing samples for researchers and detection developers*. URL: https://github.com/rf-peixoto/phishing_5Fpot.

[37] S. Pichai. *An important next step on our AI journey*. Feb. 2023. URL: https://blog.google/technology/ai/bard-google-ai-search-updates/.

[38] A. S. Project. *Apache SpamAssassin Public Corpus*. 2005. URL: https://spamassassin.apache.org/old/publiccorpus/.

[39] D. Radev. *CLAIR collection of fraud email (Repository)*. June 2008. URL: https://aclweb.org/aclwiki/CLAIR_collection_of_fraud_email_(Repository).

[40] S. S. Roy, P. Thota, K. V. Naragam, and S. Nilizadeh. "From Chatbots to Phishbots?: Phishing Scam Generation in Commercial Large Language Models". In: *2024 IEEE Symposium on Security and Privacy (SP)*. 2024, pp. 36–54. DOI: 10.1109/SP54263.2024.00182.

[41] G. Sakkis, I. Androutsopoulos, G. Paliouras, V. Karkaletsis, C. D. Spyropoulos, and P. Stamatopoulos. "A Memory-Based Approach to Anti-Spam Filtering for Mailing Lists". In: *Information Retrieval*, 6(1) (Jan. 2003), pp. 49–73. ISSN: 1573-7659. DOI: 10.1023/a:1022948414856. URL: http://dx.doi.org/10.1023/A:1022948414856.

[42] G. Sanderson. *Attention in transformers, step-by-step | Deep Learning Chapter 6*. Ed. by J. Sun. Apr. 2024. URL: https://www.3blue1brown.com/lessons/attention.

[43] R. Sennrich, B. Haddow, and A. Birch. *Neural Machine Translation of Rare Words with Subword Units*. 2016. arXiv: 1508.07909 [cs.CL]. URL: https://arxiv.org/abs/1508.07909.

[44] Shubham. *Difference between Spam and Phishing Mail*. July 2025. URL: https://www.geeksforgeeks.org/computer-networks/difference-between-spam-and-phishing-mail/.

[45] Statista. *Most commonly reported cybercrime categories in the United States in 2024, by number of individuals affected*. 2025. URL: https://www.statista.com/statistics/184083/commonly-reported-types-of-cyber-crime-us/ (visited on 09/30/2025).

[46] G. Team et al. *Gemini: A Family of Highly Capable Multimodal Models*. 2025. arXiv: 2312.11805 [cs.CL]. URL: https://arxiv.org/abs/2312.11805.

[47] T. toimisto. *Tietojen kalastelu | Tietosuojavaltuutetun toimisto*. URL: https://tietosuoja.fi/tietojenkalastelu.

[48] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. 2023. arXiv: 2307.09288 [cs.CL]. URL: https://arxiv.org/abs/2307.09288.

[49] TREC. *Text REtrieval Conference (TREC) Spam Track*. 2005-2007 public spam corpora. Feb. 2007. URL: https://trec.nist.gov/data/spam.html.

[50] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. "Attention is All you Need". In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc., 2017. URL: https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[51] N. T. Vy Nguyen, F. D. Childress, and Y. Yin. "Debate-Driven Multi-Agent LLMs for Phishing Email Detection". In: *2025 13th International Symposium on Digital Forensics and Security (ISDFS)*. IEEE, Apr. 2025, pp. 1–5. DOI: 10.1109/isdfs65363.2025.11012014. URL: http://dx.doi.org/10.1109/ISDFS65363.2025.11012014.

[52] *Weekly review of the National Cyber Security Centre Finland (NCSC-FI) - 37/2025*. Sept. 2025. URL: https://www.kyberturvallisuuskeskus.fi/en/news/weekly-review-national-cyber-security-centre-finland-ncsc-fi-372025?toggle=Phishing+messages+sent+in+the+name+of+S-Bank.

[53] M. Weinz, N. Zannone, L. Allodi, and G. Apruzzese. "The Impact of Emerging Phishing Threats: Assessing Quishing and LLM-generated Phishing Emails against Organizations". In: *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. ASIA CCS '25. ACM, Aug. 2025, pp. 1550–1566. DOI: 10.1145/3708821.3736195. URL: http://dx.doi.org/10.1145/3708821.3736195.

[54] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt. *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. 2023. arXiv: 2302.11382 [cs.SE]. URL: https://arxiv.org/abs/2302.11382.

[55] Y. Xue, E. Spero, Y. S. Koh, and G. Russello. *MultiPhishGuard: An LLM-based Multi-Agent System for Phishing Email Detection*. 2025. arXiv: 2505.23803 [cs.CR]. URL: https://arxiv.org/abs/2505.23803.